PhishGuard

Fine-Tuned BERT for Phishing Email Detection

Arad Fadaei, Mahboobeh Yasini

Version 1.0: Final Submission

Table of Contents

1. Problem . . . 2
2. Objective of the Project . . . 3
3. Details of the Technical Solution Implemented . . . 4
   3.1. System Overview . . . 4
   3.2. Data Sources and Dataset Construction . . . 4
   3.3. Exploratory Data Analysis . . . 5
   3.4. Preprocessing Pipeline . . . 7
   3.5. Model Architecture . . . 7
   3.6. Training Strategy . . . 8
   3.7. Evaluation Method . . . 8
   3.8. Final Results . . . 9
   3.9. Inference and Explainability . . . 10
   3.10. User Interface and Deployment Path . . . 10
4. Lessons Learned and Observations . . . 11
5. Conclusion . . . 12
6. AI Assistance Disclosure . . . 13

***This report presents PhishGuard, a phishing email detection system built** by fine-tuning* `bert-base-uncased` *for binary classification. The project combines data preparation, preprocessing, model training, evaluation, explainable inference, and a Gradio-based web interface into one reproducible pipeline. On the held-out test split, the final model achieved 99.48% accuracy, 0.9948 macro F1, and 0.9998 ROC-AUC. The result shows that a compact, well-engineered transformer pipeline can distinguish legitimate email from phishing attempts with very high reliability while remaining practical to deploy and inspect.*

Chapter 1. Problem

Email remains one of the most common attack surfaces for social engineering. Phishing messages are inexpensive to produce, easy to distribute at scale, and increasingly difficult to distinguish from normal business communication. The main technical challenge is that modern phishing emails do not always rely on obvious spelling errors or suspicious formatting. Many imitate ordinary workplace requests, account notifications, invoice workflows, or urgent administrative actions.

Traditional email filtering approaches often depend on blacklists, hand-written rules, sender reputation, or shallow keyword matching. Those methods are useful, but they become less effective when an attacker changes wording, rotates infrastructure, or deliberately mimics legitimate organizational tone. In other words, the classification problem is semantic rather than purely lexical. A model must learn patterns of intent, urgency, impersonation, and deception that appear in natural language, even when the message looks professionally written.

This project addresses that gap by treating phishing detection as a supervised natural language processing task. Instead of relying on manually engineered rules alone, the system learns from labeled email corpora and predicts whether an email is legitimate or phishing based on the message content.

Chapter 2. Objective of the Project

The project had four main objectives.

1. Build an end-to-end phishing email detection pipeline that is reproducible and easy to run.
2. Fine-tune a transformer model that can classify email text as legitimate or phishing with strong generalization on held-out data.
3. Preserve enough interpretability to inspect which tokens influenced a prediction instead of returning only a hard label.
4. Package the result so it can be used both from the command line and through a lightweight web application.

The success criterion was not only high predictive performance, but also a clean engineering workflow: raw data ingestion, preprocessing, stratified splitting, model checkpointing, final evaluation, and user-facing inference.

Chapter 3. Details of the Technical Solution Implemented

3.1. System Overview

PhishGuard is implemented as a modular Python project built around Hugging Face Transformers, PyTorch, pandas, and scikit-learn. The repository separates the pipeline into dedicated modules for data loading, text preprocessing, dataset preparation, training, evaluation, inference, and web serving. This structure allowed the project to remain testable and maintainable while still being simple to run through the scripts under `src/scripts/`.

At a high level, the implemented workflow is:

1. Load labeled email data from the configured source file.
2. Clean and normalize the text.
3. Split the dataset into train, validation, and test partitions using stratification.
4. Tokenize the text for BERT.
5. Fine-tune the classifier with early stopping and checkpoint management.
6. Evaluate the best checkpoint on the held-out test set.
7. Serve predictions through a CLI tool and a Gradio web interface.

3.2. Data Sources and Dataset Construction

The repository contains seven raw email corpora under `data/raw/`.

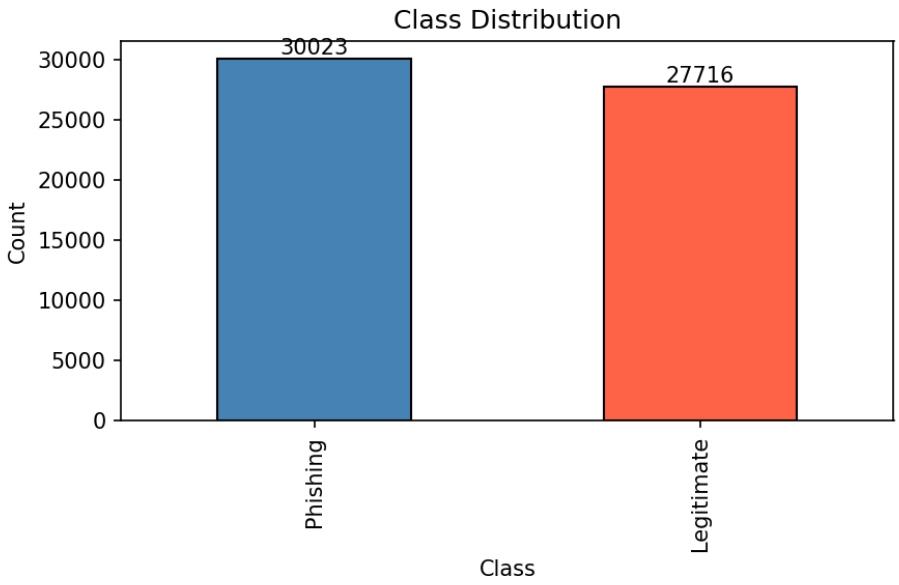
Dataset file	Role in the project
`CEAS_08.csv`	Public email security corpus used as part of the overall data collection.
`Enron.csv`	Legitimate enterprise email corpus used to represent normal business communication.
`Ling.csv`	Phishing-related corpus used to broaden attacker language patterns.
`Nazario.csv`	Phishing-related corpus that contributes additional malicious examples.
`Nigerian_Fraud.csv`	Fraud-oriented email corpus that adds persuasive scam language and urgency patterns.
`phishing_email.csv`	The consolidated primary file used by the training pipeline through `config.yaml`.
`SpamAssasin.csv`	Auxiliary email corpus included in the project data inventory.

Although the repository stores several raw corpora, the implemented training script is configured to load `phishing_email.csv` as the primary source file. This design keeps the training path straightforward while still preserving the broader dataset collection inside the repository for traceability and future extension.

After preprocessing, the dataset is split with a stratified 70/15/15 ratio into train, validation, and test sets. The saved processed splits contain 82,485 examples in total.

Split	Rows	Legitimate	Phishing
Train	57,739	27,716	30,023
Validation	12,372	5,939	6,433
Test	12,374	5,940	6,434

The class distribution is close to balanced, which is important because it reduces the risk that the classifier learns to exploit a majority-class shortcut.



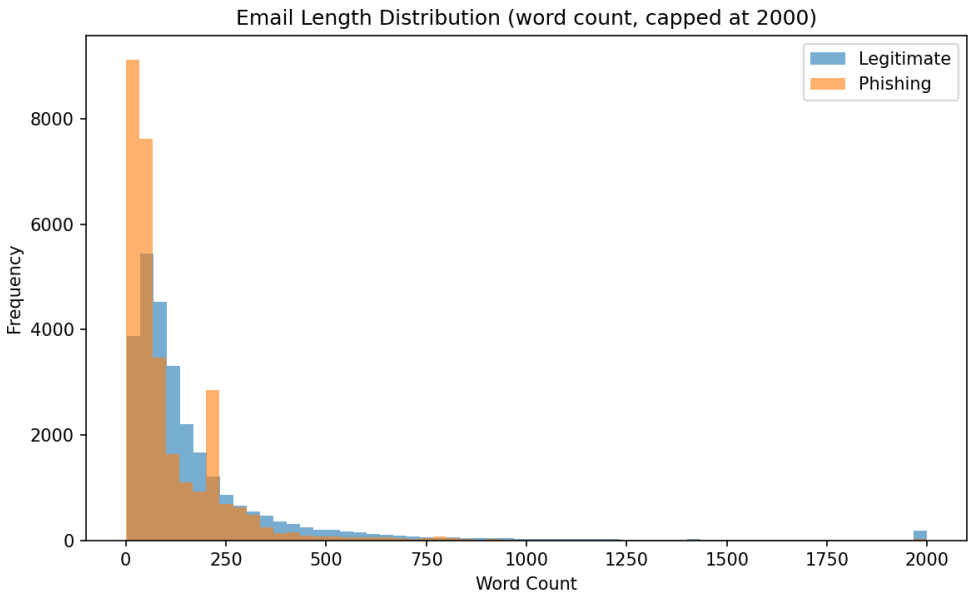
3.3. Exploratory Data Analysis

Exploratory data analysis was included to validate the dataset before training and to understand broad linguistic differences between legitimate and phishing messages. The project generates several reusable figures:

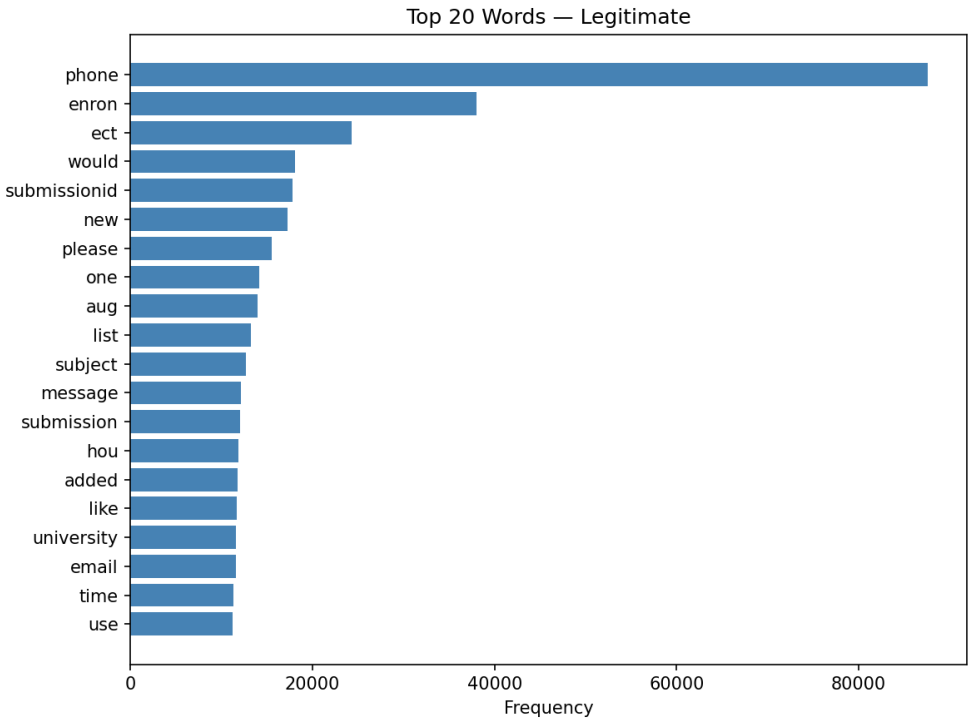
1. Class distribution
2. Text length distribution by class
3. Top words in legitimate email
4. Top words in phishing email
5. Word clouds for both classes
6. URL presence rate

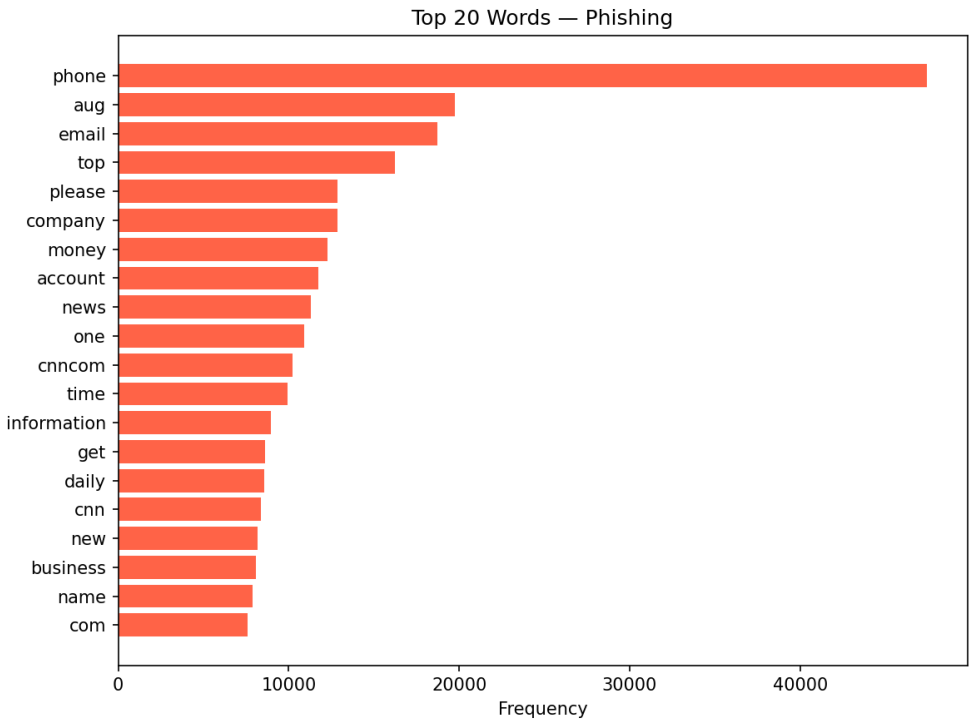
The length distribution shows that email bodies vary significantly in size, which justifies the use of

truncation at a fixed maximum sequence length during tokenization.



The word-level plots and word clouds are not used directly by the model, but they provide a qualitative check that the dataset contains plausible language cues for both benign and malicious communication.





3.4. Preprocessing Pipeline

Raw email text is noisy. It may contain HTML fragments, URLs, repeated whitespace, and personally identifiable information such as phone numbers or email addresses. If that noise is left untouched, the model may overfit to incidental tokens rather than genuine phishing cues.

The implemented preprocessing pipeline applies four operations in a fixed order:

1. **HTML stripping** using BeautifulSoup, with a regex fallback if BeautifulSoup is unavailable.
2. **URL normalization** by replacing links with the placeholder token `[URL]`.
3. **PII masking** by replacing email addresses with `[EMAIL]` and phone numbers with `[PHONE]`.
4. **Whitespace normalization** to collapse repeated spaces and line breaks.

This design keeps the semantic structure of the email while reducing unnecessary variation. For example, two phishing emails containing different domains still share the same normalized `[URL]` token, allowing the model to focus on the surrounding language rather than memorizing a particular string.

3.5. Model Architecture

The core classifier is based on `bert-base-uncased`, a bidirectional transformer pre-trained on large general-domain corpora. The project wraps it with Hugging Face `BertForSequenceClassification` configured for two output labels:

1. `0` for legitimate email
2. `1` for phishing email

The maximum sequence length is set to 256 tokens. This is a practical tradeoff between retaining

enough content from typical email bodies and keeping GPU memory usage under control. The final classification layer maps the contextualized BERT representation to a binary decision.

One implementation detail that improved efficiency is the custom `PhishingDataset` class. Instead of tokenizing every sample repeatedly on each epoch, the dataset pre-tokenizes the text once during initialization. That reduces CPU overhead during training and makes better use of the GPU.

3.6. Training Strategy

Training is handled through Hugging Face `Trainer`, which provides a robust loop for optimization, evaluation, checkpointing, and metric tracking. The project configuration uses the hyperparameters shown below.

Hyperparameter	Value
Base model	`bert-base-uncased`
Batch size	16
Learning rate	`2.0e-5`
Epochs	3
Warmup steps	500
Weight decay	0.01
Maximum sequence length	256
Evaluation strategy	Per epoch
Checkpoint save strategy	Per epoch
Best-model selection metric	Macro F1
Mixed precision	FP16 enabled
Data loader workers	4

The training loop also uses early stopping with a patience of 2 validation checks. That prevents unnecessary over-training and keeps the selected checkpoint aligned with generalization performance rather than just continued loss minimization on the training split.

Another practical feature is automatic checkpoint recovery. If training is interrupted, the trainer scans the output directory for the latest `checkpoint-*` folder and resumes from that point. This matters when working with long-running GPU experiments.

3.7. Evaluation Method

The final model is evaluated on the held-out test set, which was never used for optimization. The evaluation module computes:

1. Accuracy
2. Macro F1 score
3. ROC-AUC

4. Full classification report
5. Confusion matrix
6. ROC curve plot

These metrics were chosen because phishing detection has asymmetric risks. Accuracy is useful at a glance, but macro F1 gives a more balanced picture of per-class quality, and ROC-AUC measures ranking quality across thresholds.

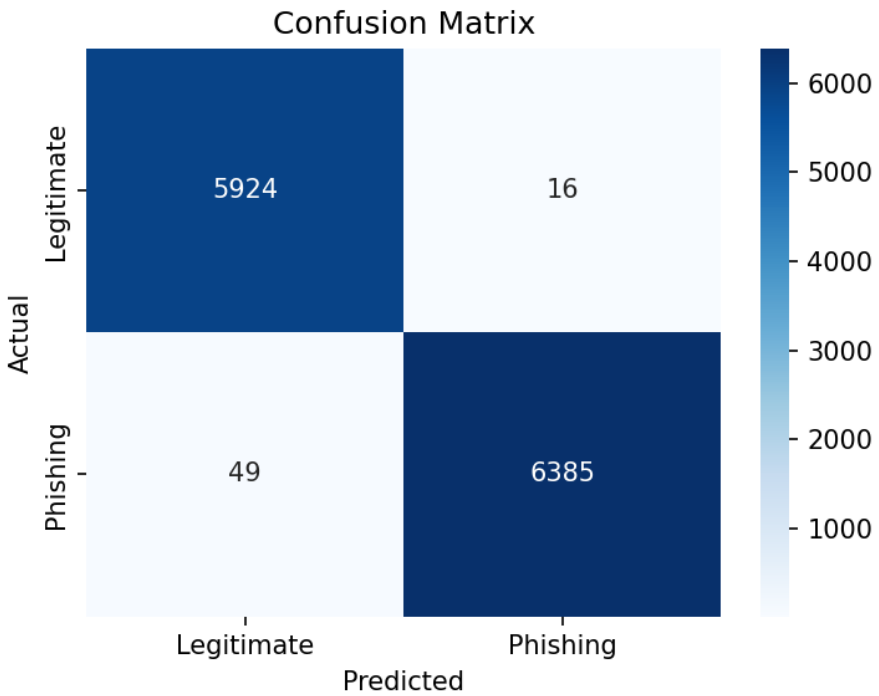
3.8. Final Results

The saved best checkpoint produced the following results on the 12,374-example test split.

Metric	Value
Accuracy	99.48%
Macro F1	0.9948
ROC-AUC	0.9998
False positives	16
False negatives	49

The confusion matrix provides a clearer operational view.

Actual class	Predicted legitimate	Predicted phishing
Legitimate	5,924	16
Phishing	49	6,385



These numbers indicate that the model learned a strong separation boundary between benign and

malicious emails. The very small number of false positives is particularly encouraging because excessive false alarms reduce trust in a security tool and create operational friction for users.

3.9. Inference and Explainability

The inference pipeline is implemented in `PhishGuardPredictor`. For a single input email, it:

1. Applies the same preprocessing pipeline used during training.
2. Tokenizes the text using the saved tokenizer.
3. Runs the fine-tuned classifier.
4. Converts logits to probabilities through softmax.
5. Returns the predicted label and confidence.
6. Extracts token-level attention scores from the last transformer layer.

The explainability mechanism is intentionally lightweight. It uses the attention from the `[CLS]` token in the final layer, averaged across heads, to estimate which tokens received the most model focus. This does not provide a perfect causal explanation, but it gives the user a practical indication of the parts of the message that influenced the prediction.

3.10. User Interface and Deployment Path

To make the model accessible beyond offline experiments, the project includes both a command-line predictor and a Gradio web interface. The Gradio application accepts raw email text and returns three outputs:

1. The predicted verdict
2. The confidence score
3. A short list of highly attended tokens

This interface is useful for demonstration, manual validation, and rapid usability testing. It also shows that the project is not only a model training exercise but a deployable NLP workflow with an end-user entry point.

Chapter 4. Lessons Learned and Observations

Several lessons became clear during the implementation.

First, **data quality mattered as much as model choice**. Even with a strong pre-trained backbone, preprocessing and label normalization were essential. If raw emails are left with inconsistent formatting, duplicated artifacts, or highly specific identifiers, the model can learn shortcuts that do not generalize.

Second, **normalization improved signal density**. Replacing URLs, email addresses, and phone numbers with stable placeholder tokens reduced vocabulary noise without erasing the fact that those elements were present. In phishing detection, the presence of a link or contact instruction is important, but the exact string is often less informative than the surrounding language.

Third, **engineering efficiency changed the practical training experience**. Pre-tokenizing the dataset, enabling FP16 mixed precision, using pinned memory, and increasing data loader workers all reduced bottlenecks. Those improvements do not change the classifier architecture, but they make the difference between a pipeline that is theoretically sound and one that is pleasant to iterate on.

Fourth, **macro F1 was a better checkpoint criterion than raw accuracy**. Because phishing detection has class-specific consequences, selecting the best model using macro F1 ensured that validation performance stayed balanced across legitimate and phishing classes.

Fifth, **simple explainability is valuable even when it is not perfect**. Security systems are easier to trust when they provide some indication of why a message was flagged. The attention-based token view gives a lightweight explanation channel without requiring a separate interpretation pipeline.

Finally, the evaluation results suggest that transformer-based text classification is a strong fit for phishing detection when the data pipeline is well controlled. The final model achieved high accuracy with only three fine-tuning epochs, which reflects the value of transfer learning from a broadly pre-trained language model.

Chapter 5. Conclusion

PhishGuard demonstrates that phishing email detection can be implemented as a clean end-to-end NLP system rather than a collection of brittle rules. The project integrates data preprocessing, stratified dataset management, transformer fine-tuning, rigorous evaluation, explainable single-message inference, and a lightweight web application in one coherent workflow.

The final model achieved 99.48% accuracy and 0.9948 macro F1 on the held-out test set, with only 16 false positives and 49 false negatives. From both a machine learning and software engineering perspective, the project met its objective: it produced a high-performing phishing classifier and packaged it in a way that supports reproducibility, inspection, and practical use.

Chapter 6. AI Assistance Disclosure

AI tools were used in a limited supporting role during the development of this project. Specifically, AI assistance was used to help accelerate implementation of the Gradio-based web interface and to help diagnose training issues during development when the model was unexpectedly running on the CPU even though CUDA was available and GPU acceleration was intended to be enabled.

The AI assistance was used for debugging guidance, implementation suggestions, and iteration speed. Final engineering decisions, code integration and execution, evaluation, and report writing remained our responsibility. All reported results were produced from the final pipeline after manual review and validation.